

Backpropagation from Scratch: A Beginner's Guide

A Deep Dive into the Micrograd Engine

1 The Core Concept: The “Paper Trail”

In standard programming, if you calculate $c = a + b$, the computer gives you the result and immediately forgets how it got there. In Deep Learning, we need the computer to remember.

Think of our `Value` object as a piece of paper that records:

1. **The Data:** The current number (e.g., 5.0).
2. **The Ancestry:** Which other pieces of paper were used to calculate this one?
3. **The Operation:** Was it addition? Multiplication?
4. **The Gradient:** How much does this specific number affect the final result?

1.1 The Chain Rule (The Secret Sauce)

The entire engine relies on the **Chain Rule**. If z depends on y , and y depends on x , then the influence of x on z is:

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$$

In our code, `self.grad` represents $\frac{dL}{d(\text{self})}$, where L is the final output.

2 The Implementation: Line-by-Line

2.1 1. The Constructor

We initialize every number with a gradient of 0. We don't know the influence yet!

```
1 class Value:
2     def __init__(self, data, _children=(), _op='', _label=''):
3         self.data = data
4         self.grad = 0.0 # Initially, this number has no known impact
5         self._prev = set(_children) # The "parents"
6         self._backward = lambda: None # Empty "recipe" for math
```

2.2 2. Addition: Passing the Influence

When we add two numbers, $out = a + b$, the derivative of out with respect to a is just 1. So, a simply inherits whatever gradient out has.

```

1 def __add__(self, other):
2     other = other if isinstance(other, Value) else Value(other)
3     out = Value(self.data + other.data, (self, other), '+')
4
5     def _backward():
6         self.grad += 1.0 * out.grad
7         other.grad += 1.0 * out.grad
8     out._backward = _backward
9     return out

```

Note: We use += because if a variable is used twice, its gradients should accumulate (add up).

2.3 3. Multiplication: The Switcheroo

If $out = a \cdot b$, the derivative with respect to a is b .

```

1 def __mul__(self, other):
2     other = other if isinstance(other, Value) else Value(other)
3     out = Value(self.data * other.data, (self, other), '*')
4
5     def _backward():
6         self.grad += other.data * out.grad
7         other.grad += self.data * out.grad
8     out._backward = _backward
9     return out

```

3 The “Brain” of the Operation: Topological Sort

We cannot calculate a parent’s gradient until we have calculated the child’s gradient. To do this, we lay all our “pieces of paper” out in a line such that every parent comes **after** its children. Then, we walk through that line backwards.

4 Full Code Listing

```

1 import math
2
3 class Value:
4     def __init__(self, data, _children=(), _op='', label=''):
5         self.data = data
6         self.grad = 0.0
7         self._backward = lambda: None
8         self._prev = set(_children)
9         self._op = _op
10        self.label = label
11
12    def __add__(self, other):
13        other = other if isinstance(other, Value) else Value(other)
14        out = Value(self.data + other.data, (self, other), '+')
15        def _backward():
16            self.grad += 1.0 * out.grad
17            other.grad += 1.0 * out.grad
18        out._backward = _backward
19        return out
20
21    def __mul__(self, other):

```

```

22     other = other if isinstance(other, Value) else Value(other)
23     out = Value(self.data * other.data, (self, other), '*')
24     def _backward():
25         self.grad += other.data * out.grad
26         other.grad += self.data * out.grad
27     out._backward = _backward
28     return out
29
30 def tanh(self):
31     x = self.data
32     t = (math.exp(2*x) - 1)/(math.exp(2*x) + 1)
33     out = Value(t, (self, ), 'tanh')
34     def _backward():
35         self.grad += (1 - t**2) * out.grad
36     out._backward = _backward
37     return out
38
39 def backward(self):
40     topo = []
41     visited = set()
42     def build_topo(v):
43         if v not in visited:
44             visited.add(v)
45             for child in v._prev:
46                 build_topo(child)
47             topo.append(v)
48     build_topo(self)
49     self.grad = 1.0
50     for node in reversed(topo):
51         node._backward()

```